

CSE 396 — Computer Engineering Project

Group 10 | Assignment 2

Module Interface & Header Design

LIDAR-Guided Hospital Navigation Robot

Date	29 March 2026
Course	CSE 396 — Computer Engineering Project
Assignment	Assignment 2 — Module Interface & Header Design

#	Name	Primary Module	Secondary Module(s)
1	Muhammet Gökhan ÖZBEK	<i>MOD-01 – Voice Communication</i>	<i>MOD-06 – Sensor Integration</i>
2	Hicran KOÇ	<i>MOD-03 – Hardware & Motor Control</i>	<i>MOD-04 – SLAM</i>
3	Fatih Emre OĞAN	<i>MOD-04 – SLAM (co-primary)</i>	<i>MOD-05 – NAV</i>
4	Samet ALKAN	<i>MOD-02 – Digital Twin</i>	<i>MOD-05 – NAV</i>
5	Mervan Deniz YILDIRIM	<i>MOD-04 – SLAM (co-primary)</i>	<i>MOD-02, MOD-03</i>
6	Ertuğrul ERGÜL	<i>MOD-05 – NAV (co-primary)</i>	<i>MOD-04 – SLAM</i>
7	Sayed Khalilullah HASHIMI	<i>MOD-06 – Sensor Integration</i>	<i>MOD-03 – Hardware</i>
8	Ömer Faruk GÜRSEL	<i>MOD-05 – NAV (co-primary)</i>	<i>MOD-06 – Sensor Integration</i>

1. Overview & Motivation

The LIDAR-Guided Hospital Navigation Robot is developed by eight engineers split across six independently owned software and hardware modules. Each module is built in relative isolation and must ultimately cooperate in real time — the robot cannot guide a visitor if the navigation engine does not know what the sensor module calls "too close", or if the SLAM module publishes a pose struct whose fields the NAV module does not recognise.

This document is the team's first formal engineering contract. It answers three questions that must be settled before any coding begins:

- What data structures flow between modules, and what do every field name and unit mean?
- Which functions does each module expose, what are their exact signatures, and what do they return on failure?
- Who is responsible for each module, and who must be notified when an interface changes?

A clear, thorough interface specification prevents three categories of costly failure: (1) integration surprises in Week 7 when sub-systems are first combined; (2) silent data corruption caused by conflicting assumptions about units or byte order; and (3) wasted rework when one team changes an internal type and another team's code breaks without warning.

Interfaces will evolve as the project progresses — that is expected and normal. What matters is that every change is communicated to all dependent modules, reflected in a versioned header file, and recorded in this document's communication table.

Engineering principle: *"Define the interfaces before you write a single line of code. If two teams disagree about what a function returns, no amount of clever implementation will fix the integration."*

2. Module–Student Assignment Table

The table below shows the official ownership and support assignments. Primary members lead and are ultimately responsible for their module's interface; Secondary members contribute and provide cross-module support.

Module ID	Module Name	Primary Student(s)	Secondary Student(s)	Brief Role
MOD-01	Voice Communication	Muhammet Gökhan ÖZBEK	Hicran KOÇ, Fatih Emre OĞAN	Speech-to-text, TTS, goal command dispatch
MOD-02	Digital Twin & Visualization	Samet ALKAN	Gökhan ÖZBEK, Mervan YILDIRIM	Real-time 3-D robot state display; floor-map overlay
MOD-03	Hardware & Motor Control	Hicran KOÇ	Mervan YILDIRIM, Sayed HASHIMI	PWM drive, encoder odometry, IMU, power management
MOD-04	SLAM – Mapping & Localization	Fatih Emre OĞAN Mervan Deniz YILDIRIM	Hicran KOÇ, Ertuğrul ERGÜL	LiDAR-based occupancy map; real-time pose estimation
MOD-05	NAV – Path Planning & Control	Ertuğrul ERGÜL Ömer Faruk GÜRSEL	Fatih Emre OĞAN, Samet ALKAN	A* planner, waypoint follower, obstacle FSM, re-routing
MOD-06	Sensor Integration & Control	Sayed Khalilullah HASHIMI	Gökhan ÖZBEK, Ömer Faruk GÜRSEL	Ultrasonic/IR sensors, calibration, unified sensor API

Note: MOD-04 and MOD-05 each have two co-primary owners due to their higher complexity. Both co-primaries are jointly responsible for all deliverables of their module.

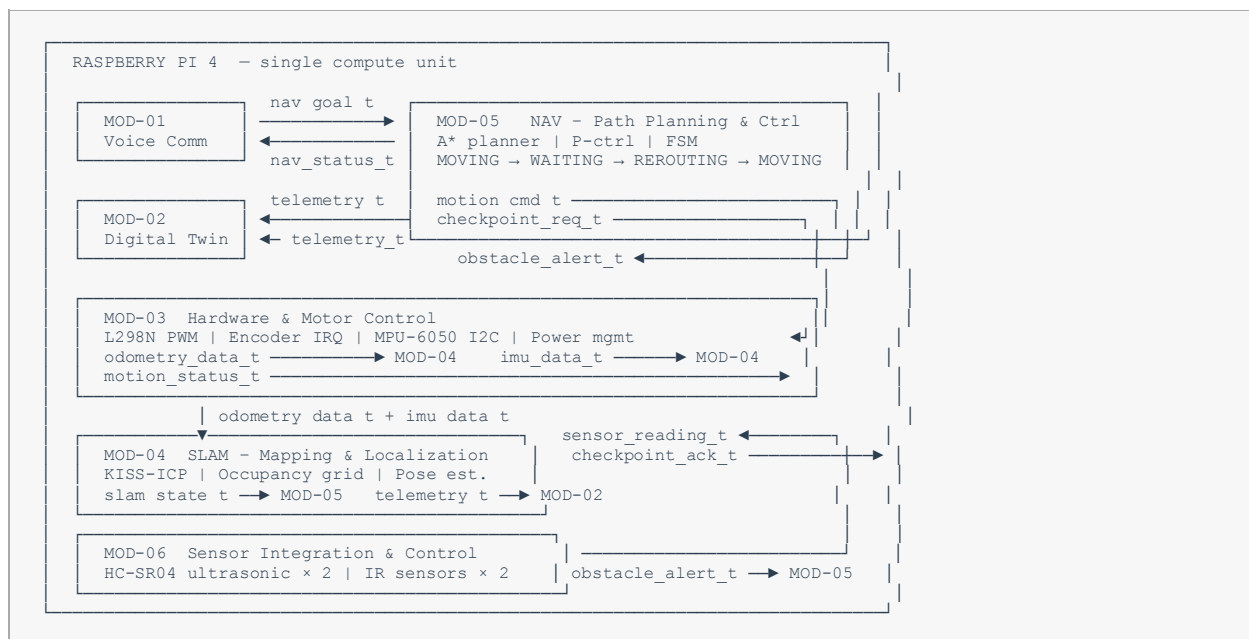
3. System & Module Overview

3.1 System Architecture

The robot is a differential-drive mobile platform whose single compute unit — a Raspberry Pi 4 — runs all six software modules. A RPLidar A2 provides 360° distance scans for simultaneous SLAM localisation and obstacle detection. An MPU-6050 IMU feeds angular-rate data for heading fusion, and a pair of quadrature wheel encoders supply odometry pulses. Motor output is delivered through an L298N H-bridge to two DC gear-motors. Voice interaction uses a USB microphone and a mini speaker/amplifier pair. A PC-resident Digital Twin mirrors the robot state in real time via TCP WebSocket.

3.2 Module Block Diagram

Arrows indicate the primary direction of data flow. All intra-RPi communication uses thread-safe queue.Queue or shared memory with mutex; the Digital Twin link uses TCP WebSocket JSON.



3.3 Module Descriptions

MOD-01 – Voice Communication

Captures audio from the USB microphone, performs on-device speech-to-text (STT) using Whisper or Vosk, parses the recognised utterance into a structured `nav_goal_t`, and dispatches that goal to MOD-05. In the opposite direction it receives `nav_status_t` from MOD-05 and synthesises audible Turkish speech via Piper TTS (`tr_TR-dfki-medium`).

MOD-02 – Digital Twin & Visualization

Subscribes to `telemetry_t` from MOD-04/MOD-05 via TCP WebSocket (navigation state is carried in the `telemetry_t.nav` field), and renders them on a 3-D floor map inside Unity or Gazebo. Implements a priority-based pin-drop overlay (up to 32 pins) and exports a WebSocket feed for a browser-based dashboard (default port 8765).

MOD-03 – Hardware & Motor Control

Owns all low-level hardware interfacing: generates PWM signals for the L298N H-bridge (50 Hz), reads quadrature encoder pulses via GPIO interrupts, integrates MPU-6050 IMU data over I²C (400 kHz), and manages power distribution. Publishes `odometry_data_t` (50 Hz) and `imu_data_t` (100 Hz) to MOD-04, and `motion_status_t` (50 Hz) to MOD-05. Receives `motion_cmd_t` from MOD-05.

MOD-04 – SLAM – Mapping & Localization

Runs the KISS-ICP LiDAR odometry pipeline, fusing RPLidar A2 scans with odometry and IMU priors from MOD-03. Maintains a 2-D occupancy grid (5 cm/cell) and continuously publishes `slam_state_t` to MOD-05 (20 Hz). Handles drift correction at predefined corridor-junction checkpoints via scan-to-reference matching (confidence ≥ 0.80) through a `checkpoint_req_t` / `checkpoint_ack_t` handshake with MOD-05.

MOD-05 – NAV – Path Planning & Control

Consumes the occupancy map and current pose from MOD-04 and a `nav_goal_t` from MOD-01 to compute an A* waypoint sequence. A proportional heading controller ($K_p = 1.2$) converts heading error into `motion_cmd_t` sent to MOD-03 at 50 Hz. Implements the MOVING $\rightarrow$ WAITING $\rightarrow$ REROUTING $\rightarrow$ MOVING obstacle FSM consuming `obstacle_alert_t` from MOD-06, and publishes `nav_status_t` to MOD-01 on every state transition. Navigation state reaches MOD-02 via the `telemetry_t.nav` field.

MOD-06 – Sensor Integration & Control

Manages all onboard sensors beyond the primary LiDAR: up to 2 HC-SR04 ultrasonic rangefinders and 2 IR obstacle sensors. Responsible for placement, calibration, and noise filtering. Publishes `sensor_reading_t` to MOD-04 (10 Hz, shared memory) and, when any sensor crosses the 0.50 m obstacle threshold, event-publishes `obstacle_alert_t` to MOD-05 via `queue.Queue`.

4. Hardware & Platform Summary

4.1 Target Hardware

Component	Model / Part	Role	Owner
Main Computer	Raspberry Pi 4 (2 GB+)	Runs all software modules; sensor I/O, planning, TTS/STT	All
2-D LiDAR	RPLidar A2 (SLAMTEC)	360° scan for SLAM, obstacle detection, checkpoint matching	MOD-04/05
IMU	MPU-6050 (I2C, 0x68)	3-axis gyro + accel; heading fusion — complementary filter $\alpha=0.98$	MOD-03
Wheel Encoders	Quadrature encoders $\times 2$	Pulse-count odometry; 20 CPR each	MOD-03
Motor Driver	L298N H-Bridge	PWM-driven bidirectional DC motor control via GPIO	MOD-03
Drive Motors	DC Gear Motors $\times 2$	Differential-drive locomotion (12 V)	MOD-03
Battery	LiPo 11.1 V / 2200 mAh	Primary power supply via UBEC (5 V to RPi)	MOD-03
Microphone	Youmi Mini USB 2.0	16 kHz, 16-bit mono; ALSA default capture device	MOD-01
Speaker + Amp	MakerHawk + DROK PAM8406	TTS audio output; 44.1 kHz via ALSA	MOD-01
Aux. Sensors	HC-SR04 $\times 2$, IR sensors $\times 2$	Short-range proximity; supplements LiDAR blind zones	MOD-06

4.2 Communication Buses

Bus	Devices	Speed / Params	Modules
I2C (400 kHz)	MPU-6050 IMU	Addr 0x68; fast-mode; sole I2C master = MOD-03	MOD-03 $\leftrightarrow$ MOD-04
GPIO PWM	L298N IN1–IN4 + ENA/ENB	50 Hz carrier; 3.3 V logic; RPi BCM pins	MOD-03
GPIO IRQ	Encoder channels A/B $\times 2$	Rising-edge interrupt; BCM pins 17,18,22,23	MOD-03
USB (Serial)	RPLidar A2	115200 baud; SLAMTEC SDK / /dev/ttyUSB0	MOD-04
USB Audio	Youmi Microphone (UAC)	16 kHz, 16-bit mono; ALSA hw:1,0	MOD-01
I2S / ALSA	PAM8406 amplifier	44.1 kHz stereo; ALSA / USB soundcard	MOD-01

TCP/IP (LAN)	Digital Twin (PC ↔ RPi)	WebSocket JSON; ~10 Hz; port 8765	MOD-02 ↔ MOD-04/05
GPIO (digital)	HC-SR04 TRIG/ECHO; IR OUT	3.3 V GPIO; HC-SR04 echo timeout 30 ms	MOD-06

4.3 Software Platform & External Libraries

Library / Tool	Details
OS / Runtime	Raspberry Pi OS 64-bit (Bookworm) Python 3.11 pip + virtualenv pytest
LiDAR SDK	rplidar-roboticia Python wrapper KISS-ICP (kiss-icp-ros2 or pure Python port)
Navigation	A* implementation (custom) BehaviorTree.CPP or py_trees for obstacle FSM
STT	OpenAI Whisper (tiny model, on-device) or Vosk offline
TTS	Piper TTS — tr_TR-dfki-medium model ALSA / PyAudio playback
Digital Twin	Gazebo (primary) or Unity 2022 LTS python-socketio for WebSocket telemetry
IMU Driver	smbus2 custom complementary filter ($\alpha = 0.98$)
Build / VCS	Git (GitHub) pip + virtualenv pytest for unit tests

5. Inter-Module Communication

All inter-module data exchange within the Raspberry Pi process is implemented as Python dataclasses / C structs passed via thread-safe queues (`queue.Queue`) or shared memory with a mutex. The Digital Twin telemetry link uses a TCP WebSocket. The table below is the authoritative interface contract — all header files must match it exactly.

5.1 Communication Table

From	To	Data / Signal	Type	Dir	Notes / Frequency
MOD-03	MOD-04	odometry_data_t	struct	03→04	50 Hz; left/right pulse counts, velocity
MOD-03	MOD-04	imu_data_t	struct	03→04	100 Hz; gyro θ , accel xyz, timestamp
MOD-04	MOD-05	slam_state_t	struct	04→05	20 Hz; x, y, θ , confidence
MOD-04	MOD-05	occ_map_t	struct	04→05	On map update; 2D grid, resolution, origin
MOD-04	MOD-02	telemetry_t	struct	04→02	5 Hz; pose + map snapshot for rendering
MOD-01	MOD-05	nav_goal_t	struct	01→05	On voice command; destination ID + label
MOD-05	MOD-01	nav_status_t	struct	05→01	On state change; state enum + ETA
MOD-05	MOD-03	motion_cmd_t	struct	05→03	50 Hz; v_left, v_right PWM targets
MOD-05	MOD-02	telemetry_t	struct	05→02	10 Hz; pose + nav status + battery via WebSocket
MOD-06	MOD-05	obstacle_alert_t	struct	06→05	20 Hz; distances per sensor zone
MOD-06	MOD-04	sensor_reading_t	struct	06→04	10 Hz; IR / ultrasonic filtered readings
MOD-03	MOD-05	motion_status_t	struct	03→05	50 Hz; actual v_left, v_right, stall flag
MOD-05	MOD-04	checkpoint_req_t	struct	05→04	On junction arrival; request pose snap
MOD-04	MOD-05	checkpoint_ack_t	struct	04→05	On snap complete; success/fail + snapped pose

5.2 Shared Data Type Definitions — `project_types.h`

All types below must be declared in `project_types.h`, a shared header included by every module. Each module-specific header imports this file and may define additional module-local types. Centralising shared types in one file prevents redeclaration conflicts and guarantees consistency across all modules.

`nav_goal_t` (MOD-01 → MOD-05)

```
typedef struct {
    char    destination[32]; /**< Human-readable name of target location */
    uint8_t waypoint_id;    /**< Mapped waypoint index on occupancy grid */
    uint32_t timestamp_ms;  /**< Epoch ms when goal was issued */
} nav_goal_t;
```

`nav_state_e` + `nav_status_t` (MOD-05 → MOD-01, MOD-02)

```
typedef enum {
    NAV_STATE_IDLE      = 0,
    NAV_STATE_MOVING    = 1,
    NAV_STATE_WAITING   = 2,
    NAV_STATE_REROUTING = 3,
    NAV_STATE_ARRIVED   = 4,
    NAV_STATE_RETURNING = 5
} nav_state_e;

typedef struct {
    nav_state_e state;
    char    message[64]; /**< Human-readable status string for TTS */
    uint32_t timestamp_ms;
} nav_status_t;
```

`odometry_data_t` (MOD-03 → MOD-04)

```
typedef struct {
    int32_t left_ticks;    /**< Encoder ticks since last sample (signed) */
    int32_t right_ticks;
    float   left_speed_mps; /**< Computed wheel speed (m/s) */
    float   right_speed_mps;
    uint32_t timestamp_ms;
} odometry_data_t;
```

`imu_data_t` (MOD-03 → MOD-04)

```
typedef struct {
    float   gyro_z_dps;    /**< Yaw rate degrees/s from MPU-6050 */
    float   accel_x_mss;   /**< Forward acceleration m/s2 */
    float   accel_y_mss;   /**< Lateral acceleration m/s2 */
    float   temp_c;        /**< Die temperature (°C) */
    uint32_t timestamp_ms;
} imu_data_t;
```

`slam_state_t` (MOD-04 → MOD-05, MOD-02)

```
typedef struct {
    float   x_m;           /**< Robot X position in map frame (metres) */
    float   y_m;           /**< Robot Y position in map frame */
    float   theta_rad;     /**< Heading in map frame (radians, CCW) */
    float   confidence;    /**< ICP match confidence 0.0-1.0 */
    uint8_t map_updated;   /**< 1 if occupancy grid changed this cycle */
    uint32_t timestamp_ms;
} slam_state_t;
```

`occ_map_t` (MOD-04 → MOD-05)

```
typedef struct {
    uint8_t *grid;         /**< Row-major grid; 0=free,100=occ,255=unknown */
    uint16_t width_cells;  /**< Grid width in cells */
    uint16_t height_cells; /**< Grid height in cells */
}
```

```

float    resolution_m;    /**< Cell size in metres (SLAM_GRID_RES = 0.05) */
float    origin_x_m;      /**< Map origin X in world frame          */
float    origin_y_m;      /**< Map origin Y in world frame          */
uint32_t timestamp_ms;
} occ_map_t;

```

motion_cmd_t (MOD-05 → MOD-03)

```

typedef struct {
float    left_pwm;        /**< Left motor duty cycle  -1.0 to +1.0 */
float    right_pwm;       /**< Right motor duty cycle -1.0 to +1.0 */
uint8_t  stop;            /**< 1 = immediate stop (safety override) */
uint32_t timestamp_ms;
} motion_cmd_t;

```

motion_status_t (MOD-03 → MOD-05)

```

typedef struct {
float    actual_left_mps;  /**< Actual left wheel speed (m/s) */
float    actual_right_mps; /**< Actual right wheel speed (m/s) */
uint8_t  stall_flag;      /**< 1 = encoder stall detected */
uint32_t timestamp_ms;
} motion_status_t;

```

sensor_type_e + sensor_reading_t (MOD-06 → MOD-04)

```

typedef enum {
    SENSOR_TYPE_ULTRASONIC = 0,
    SENSOR_TYPE_IR_OBSTACLE = 1
} sensor_type_e;

typedef struct {
    sensor_type_e type;
    uint8_t sensor_id; /**< 0=front-left,1=front-right,... */
    float distance_m;  /**< Measured distance (m); NaN if invalid */
    uint8_t obstacle;  /**< 1 if distance < SENSOR_OBSTACLE_THRESH_M */
    uint32_t timestamp_ms;
} sensor_reading_t;

```

obstacle_alert_t (MOD-06 → MOD-05)

```

typedef struct {
uint8_t active;          /**< 1 = obstacle in danger zone */
float    min_distance_m;  /**< Closest detected object distance (m) */
float    bearing_deg;     /**< Bearing of closest object (robot frame) */
uint32_t timestamp_ms;   /**< NaN bearing = uncertainty (see RISK-08) */
} obstacle_alert_t;

```

checkpoint_req_t + checkpoint_ack_t (MOD-05 ↔ MOD-04)

```

typedef enum { CP_REQ_SNAP = 0 } checkpoint_req_e;
typedef struct {
    checkpoint_req_e type;
    uint8_t checkpoint_id; /**< Predefined junction ID */
} checkpoint_req_t;

typedef enum { CP_ACK_OK = 0, CP_ACK_FAIL = -1 } checkpoint_ack_e;
typedef struct {
    checkpoint_ack_e result;
    uint8_t checkpoint_id;
    slam_state_t snapped_pose; /**< Valid only when result == CP_ACK_OK */
} checkpoint_ack_t;

```

telemetry_t (MOD-04 / MOD-05 → MOD-02 — JSON over WebSocket)

```

/* Serialised as JSON; C struct shown for header documentation */

```

```
typedef struct {  
    slam_state_t  pose;  
    nav_status_t  nav;  
    float         battery_v;    /**< Battery voltage (V) */  
    uint32_t      uptime_ms;  
} telemetry_t;
```

6. Per-Module Header File Specifications

Each module must provide the header file listed below. Every header must follow the naming convention, include all mandatory sections from the assignment, and import shared types from `project_types.h`. Function bodies are implemented in the corresponding `.c` / `.py` files.

MOD-01 – Voice Communication `voice.h`

Author	Muhammet Gökhan ÖZBEK	Version	v0.1 — 2026-03-29
--------	-----------------------	---------	-------------------

Constants & Macros

Constant / Macro	Description
<code>#define VOICE_MAX_CMD_LEN 64</code>	Max characters in a recognised STT command string
<code>#define VOICE_SPEAK_TIMEOUT_MS 5000</code>	TTS synthesis timeout before forced return (ms)
<code>#define VOICE_VERSION "0.1"</code>	Header version string
<code>typedef void (*voice_status_cb_t)(const nav_status_t *st)</code>	Callback invoked on each <code>nav_status_t</code> received from MOD-05

Error / Return Code Enum

```
typedef enum {
    VOICE_OK=0, VOICE_ERR_MIC=-1, VOICE_ERR_STT=-2,
    VOICE_ERR_TTS=-3, VOICE_ERR_TIMEOUT=-4
} voice_status_t;
```

Public Function Declarations

```
/** @brief Initialise microphone and TTS engine. @return VOICE_OK on success */
voice_status_t voice_init(void);

/** @brief Start continuous STT listening thread. */
voice_status_t voice_start_listen(void);

/** @brief Stop listening thread and release audio device. */
voice_status_t voice_stop_listen(void);

/** @brief Synthesise and play text via Piper TTS.
 * @param text NUL-terminated UTF-8 string (Turkish). */
voice_status_t voice_speak(const char *text);

/** @brief Block until a new nav_goal_t is parsed from speech.
 * @param out Caller-owned nav_goal_t to fill.
 * @return VOICE OK or VOICE ERR TIMEOUT */
voice_status_t voice_get_goal(nav_goal_t *out);

/** @brief Register callback for nav status t events from MOD-05.
 * @param cb Function pointer; NULL to unregister. */
void voice_set_status_cb(voice_status_cb_t cb);
```

MOD-02 – Digital Twin & Visualization `dtwin.h`

Author	Samet ALKAN	Version	v0.1 — 2026-03-29
--------	-------------	---------	-------------------

Constants & Macros

Constant / Macro	Description
#define DTWIN_PORT_DEFAULT 8765	Default WebSocket server port
#define DTWIN_PIN_MAX 32	Maximum simultaneous pins on the map overlay
#define DTWIN_TELEMETRY_HZ 10	Telemetry push rate to connected clients (Hz)
#define DTWIN_VERSION "0.1"	Header version string

Error / Return Code Enum

```
typedef enum {
    DTWIN_OK=0, DTWIN_ERR_CONN=-1, DTWIN_ERR_ENCODE=-2, DTWIN_ERR_MAP=-3, DTWIN_ERR_PIN=-4
} dtwin_status_t;
```

Public Function Declarations

```
/** @brief Initialise WebSocket server and load floor map.
 * @param map_path Path to occupancy grid file.
 * @param port WebSocket port (use DTWIN_PORT_DEFAULT). */
dtwin_status_t dtwin_init(const char *map_path, uint16_t port);

/** @brief Push combined telemetry snapshot to connected clients.
 * @param t Pointer to caller-owned telemetry_t. */
dtwin_status_t dtwin_push_telemetry(const telemetry_t *t);

/** @brief Drop a priority pin on the map at map coordinates (x_m, y_m).
 * @param priority 0=low ... 3=critical. */
dtwin_status_t dtwin_drop_pin(float x_m, float y_m, uint8_t priority);

/** @brief Remove all pins from the map overlay. */
dtwin_status_t dtwin_clear_pins(void);

/** @brief Shut down WebSocket server and free resources. */
void dtwin_shutdown(void);
```

MOD-03 – Hardware & Motor Control hw.h

Author	Hicran KOÇ	Version	v0.1 — 2026-03-29
--------	------------	---------	-------------------

Constants & Macros

Constant / Macro	Description
#define HW_ENCODER_CPR 20	Encoder counts per revolution (both wheels)
#define HW_WHEEL_DIAMETER_M 0.065f	Wheel diameter in metres (65 mm)
#define HW_WHEELBASE_M 0.17f	Distance between drive wheel centres (170 mm)

#define HW_PWM_FREQ_HZ	50	PWM carrier frequency for L298N (Hz)
#define HW_MAX_SPEED_MPS	0.5f	Maximum wheel speed — safety cap (m/s)
#define HW_VERSION	"0.1"	Header version string

Error / Return Code Enum

```
typedef enum {
    HW_OK=0, HW_ERR_GPIO=-1, HW_ERR_I2C=-2, HW_ERR_TIMEOUT=-3
} hw_status_t;
```

Public Function Declarations

```
/** @brief Initialise GPIO, PWM, I2C and encoder interrupts. */
hw_status_t hw_init(void);

/** @brief Apply a motion command to the motors.
 * @param cmd Caller-owned motion cmd t. */
hw_status_t hw_set_motion(const motion_cmd_t *cmd);

/** @brief Immediately cut power to both motors (safety override). */
hw_status_t hw_emergency_stop(void);

/** @brief Read the latest odometry_data_t from encoder ISR.
 * @param out Caller-owned struct to fill. */
hw_status_t hw_get_odometry(odometry_data_t *out);

/** @brief Read the latest imu data t from MPU-6050 via I2C.
 * @param out Caller-owned struct to fill. */
hw_status_t hw_get_imu(imu_data_t *out);

/** @brief Read battery voltage from ADC / voltage divider.
 * @param v_out Output pointer (volts). */
hw_status_t hw_get_battery_voltage(float *v_out);
```

MOD-04 – SLAM – Mapping & Localization

slam.h

Author	Fatih Emre OĞAN / Mervan Deniz YILDIRIM	Version	v0.1 — 2026-03-29
--------	--	---------	-------------------

Constants & Macros

Constant / Macro		Description
#define SLAM_GRID_RESOLUTION_M	0.05f	Occupancy grid cell size in metres (5 cm/cell)
#define SLAM_CHECKPOINT_THRESH	0.80f	Min ICP confidence to accept checkpoint pose snap
#define SLAM_LIDAR_MAX_RANGE_M	8.0f	RPLidar A2 usable range cap (metres)
#define SLAM_UPDATE_HZ	20	Pose publish rate to MOD-05 (Hz)
#define SLAM_VERSION	"0.1"	Header version string

Error / Return Code Enum

```
typedef enum {
    SLAM_OK=0, SLAM_ERR_INIT=-1, SLAM_ERR_LIDAR=-2, SLAM_WARN_LOW_CONF=1
} slam_status_t;
```

Public Function Declarations

```
/** @brief Initialise SLAM engine and load pre-built map.
 * @param map_path Path to .pgm/.yaml occupancy map. */
slam_status_t slam_init(const char *map_path);

/** @brief Feed one control-loop tick of odometry + IMU into the SLAM pipeline.
 * @param odom Latest odometry_data_t from MOD-03.
 * @param imu Latest imu_data_t from MOD-03. */
slam_status_t slam_update(const odometry_data_t *odom, const imu_data_t *imu);

/** @brief Copy the current pose estimate.
 * @param out Caller-owned slam state t to fill. */
slam_status_t slam_get_pose(slam_state_t *out);

/** @brief Return a read-only pointer to the current occupancy grid.
 * @param grid_out Output pointer to internal grid array.
 * @param width Grid width in cells.
 * @param height Grid height in cells. */
slam_status_t slam_get_map(uint8_t **grid_out, uint16_t *width, uint16_t *height);

/** @brief Override pose — called by MOD-05 after checkpoint ack t OK. */
slam_status_t slam_reset_pose(float x m, float y m, float theta rad);

/** @brief Process a checkpoint request: snap pose via scan-to-reference matching.
 * @param req Incoming checkpoint request from MOD-05.
 * @param ack Output acknowledgement with snapped pose. */
slam_status_t slam_process_checkpoint(const checkpoint_req_t *req, checkpoint_ack_t *ack);

/** @brief Shut down SLAM pipeline and release LiDAR resources. */
void slam_shutdown(void);
```

MOD-05 – NAV – Path Planning & Control nav_pathplan.h

Author	Ertuğrul ERGÜL / Ömer Faruk GÜRSEL	Version	v0.1 — 2026-03-29
--------	------------------------------------	---------	-------------------

Constants & Macros

Constant / Macro	Description
#define NAV_MAX_WAYPOINTS 64	Maximum number of waypoints in a multi-goal mission
#define NAV_PLANNER_MAX_NODES 8192	Maximum nodes expanded per planning call
#define NAV_LOOKAHEAD_M 0.35f	Pure-pursuit look-ahead distance (m)
#define NAV_GOAL_XY_TOL_M 0.05f	Goal position tolerance (m)
#define NAV_GOAL_YAW_TOL_RAD 0.10f	Goal heading tolerance (rad)

#define NAV_STUCK_TIMEOUT_MS 3000	No-progress timeout before stuck detection (ms)
#define NAV_OBSTACLE_MARGIN_M 0.15f	Obstacle inflation margin (m)
#define NAV_MIN_CLEARANCE_M 0.20f	Minimum clearance for a valid path (m)
#define NAV_CONTROL_PERIOD_MS 50	Navigation control loop period (ms)
#define NAV_VERSION "0.1"	Header version string
typedef void (*nav_cmd_vel_cb_t)(const nav_cmd_vel_t *cmd)	Called every control cycle with latest velocity command
typedef void (*nav_state_cb_t)(nav_runtime_state_t prev, nav_runtime_state_t curr)	Called on runtime-state transition
typedef void (*nav_goal_cb_t)(uint8_t wp_index, const nav_waypoint_t *wp)	Called when a waypoint is reached

Error / Return Code Enum

```
typedef enum {
    NAV_OK=0, NAV_ERR_NOT_INIT=-1, NAV_ERR_INVALID_PARAM=-2, NAV_ERR_NO_PATH=-3, NAV_ERR_TIMEOUT=-4,
    NAV_ERR_QUEUE_FULL=-5, NAV_ERR_BUSY=-6, NAV_ERR_OBSTACLE=-7, NAV_ERR_MAP_STALE=-8
} nav_result_t;
```

Runtime State Enum

```
typedef enum {
    NAV_RUNTIME_IDLE           = 0,
    NAV_RUNTIME_PLANNING       = 1,
    NAV_RUNTIME_FOLLOWING      = 2,
    NAV_RUNTIME_AVOIDING       = 3,
    NAV_RUNTIME_STUCK           = 4,
    NAV_RUNTIME_RECOVERING     = 5,
    NAV_RUNTIME_GOAL_REACHED   = 6,
    NAV_RUNTIME_ABORTED        = 7
} nav_runtime_state_t;
```

Module-Local Types

nav_waypoint_t

```
typedef struct {
    float    x;
    float    y;
    float    yaw;
    bool     has_yaw;
} nav_waypoint_t;
```

nav_cmd_vel_t

```
typedef struct { float linear_v; float angular_w; uint32_t timestamp; } nav_cmd_vel_t;
```

nav_config_t

```
typedef struct {
    float    lookahead_m;        float    max_linear_v;        float    max_angular_w;
    float    goal_xy_tol;        float    goal_yaw_tol;        float    obstacle_margin;
    uint32_t stuck_timeout_ms;
} nav_config_t;
```


nav_progress_t

```
typedef struct {
    nav_runtime_state_t state;          uint8_t waypoints_total; uint8_t waypoints_done;
    float dist_to_goal; float path_length; uint32_t timestamp;
} nav_progress_t;
```

Public Function Declarations

```
/** @brief Initialise navigation module with optional runtime config. * @param cfg Caller-owned
nav_config_t. * @return NAV_OK on success */
nav_result_t nav_init(const nav_config_t *cfg);

/** @brief Navigate to a single waypoint.
 * @param wp Caller-owned nav waypoint_t with target pose.
 * @return NAV_OK or NAV_ERR_INVALID_PARAM */
nav_result_t nav_goto(const nav_waypoint_t *wp);

/** @brief Cancel active navigation. */
nav_result_t nav_cancel(void);

/** @brief Copy current mission progress snapshot.
 * @param out Caller-owned struct to fill. */
nav_result_t nav_get_progress(nav_progress_t *out);

/** @brief Pause active navigation.
 * @return NAV_OK on success, NAV_ERR_BUSY if not navigating.
 */
nav_result_t nav_pause(void);

/** @brief Resume paused navigation.
 * @return NAV_OK on success. */
nav_result_t nav_resume(void);

/** @brief Force replanning of current path.
 * @return NAV_OK on success, NAV_ERR_NO_PATH if replanning fails. */
nav_result_t nav_replan(void);

/** @brief Release navigation resources and stop active navigation. */
void nav_deinit(void);

/** @brief Navigate through an ordered list of waypoints. * @param wps Array of
nav_waypoint_t. * @param count Number of waypoints. */
nav_result_t nav_goto_multi(const nav_waypoint_t *wps, uint8_t count);

/** @brief Get current runtime navigation state. */
nav_runtime_state_t nav_get_state(void);

/** @brief Copy latest controller velocity command. * @param out Caller-owned
nav_cmd_vel_t to fill. */
nav_result_t nav_get_cmd_vel(nav_cmd_vel_t *out);

/** @brief Register or clear velocity command callback. */
nav_result_t nav_register_cmd_vel_cb(nav_cmd_vel_cb_t cb);

/** @brief Register or clear state transition callback. */
nav_result_t nav_register_state_cb(nav_state_cb_t cb);

/** @brief Register or clear waypoint reached callback. */
nav_result_t nav_register_goal_cb(nav_goal_cb_t cb);

/** @brief Update runtime config. */
nav_result_t nav_set_config(const nav_config_t *cfg);

/** @brief Copy current runtime config. */
nav_result_t nav_get_config(nav_config_t *out);
```

MOD-06 – Sensor Integration & Control

sensor.h

Author	Sayed Khalilullah HASHIMI	Version	v0.1 — 2026-03-29
--------	---------------------------	---------	-------------------

Constants & Macros

Constant / Macro	Description
#define SENSOR_COUNT 4	Total managed sensors (2 ultrasonic + 2 IR)
#define SENSOR Obstacle THRESH M 0.50f	Distance threshold for obstacle_alert_t activation (m)
#define SENSOR_POLL_RATE_HZ 20	Polling rate for all sensors (Hz)
#define SENSOR_MAX_RANGE_M 4.0f	Maximum valid measurement range (m)
#define SENSOR_VERSION "0.1"	Header version string

Error / Return Code Enum

```
typedef enum {
    SENSOR_OK=0, SENSOR_ERR_INIT=-1, SENSOR_ERR_TIMEOUT=-2, SENSOR_WARN_NOISE=1
} sensor_status_t;
```

Public Function Declarations

```
/** @brief Initialise all GPIO sensor pins and calibration state. */
sensor_status_t sensor_init(void);

/** @brief Read all sensors once and update internal state.
 * */
sensor_status_t sensor_update(void);

/** @brief Read the raw sample from a specific sensor by ID.
 * @param sensor_id 0 - (SENSOR_COUNT-1).
 * @param out Caller-owned sensor_reading_t to fill. */
sensor_status_t sensor_get_raw_reading(uint8_t id, sensor_reading_t *reading);

/** @brief Retrieve the latest consolidated obstacle alert.
 * @param out Caller-owned obstacle alert t (active=0 if none). */
sensor_status_t sensor_get_alert(obstacle_alert_t *out);

/** @brief Compute the best detour bearing away from nearest obstacle.
 * @param sensor_id 0 - (SENSOR_COUNT-1). */
sensor_status_t sensor_get_best_detour(float *best_bearing_deg);

/** @brief */
```

7. Known Risks & Open Questions

All items below represent unresolved interface decisions or hardware uncertainties. Each must be resolved and documented before or during Week 7 (Sub-system Integration). The responsible module lead must update the affected header file, bump the version string, and notify all dependent module owners.

ID	Level	Risk / Open Question	Mitigation / Action
RISK-01	HIGH	SLAM drift on long corridors (>20 m) <i>Responsible: MOD-04</i>	KISS-ICP may accumulate pose error beyond odometry correction capacity. Mitigation: pre-map corridor-junction checkpoints; snap pose on high-confidence ICP match (≥ 0.80). Open question: checkpoint density needed for 300–400 m ² floor — to be determined during W5 initial mapping.
RISK-02	HIGH	RPLidar A2 USB instability under load <i>Responsible: MOD-04</i>	USB bus contention (microphone + LiDAR simultaneously) may drop scan frames. Mitigation: use a powered USB hub; assign dedicated USB ports. TBD: whether RPi 4 USB 3.0 port provides sufficient bandwidth for simultaneous LiDAR + audio capture.
RISK-03	MED	Speech recognition accuracy in noisy hospital <i>Responsible: MOD-01</i>	Whisper tiny model may mis-recognise Turkish destination names. Mitigation: restricted vocabulary (< 20 destinations); voice confirmation before navigation starts. Open question: should operator-PC goal input be the default fallback?
RISK-04	MED	Motor PWM / encoder IRQ latency on non-RTOS Linux <i>Responsible: MOD-03</i>	RPi OS scheduler may jitter GPIO interrupts at high speed, degrading odometry. Mitigation: use RPi.GPIO edge-detect; limit max speed to 0.5 m/s. Evaluate pigpio daemon as higher-priority alternative.
RISK-05	MED	nav_goal_t waypoint_id mapping not finalised <i>Responsible: MOD-01, MOD-05</i>	Hospital floor plan not fully mapped; waypoint_id values cannot be assigned until the map is built. MOD-01 and MOD-05 must agree on the mapping table before W7 integration week. Interim: use a hard-coded JSON lookup file.
RISK-06	LOW	LiPo battery capacity for full navigation cycle <i>Responsible: MOD-03</i>	A 2200 mAh pack driving two motors + RPi + LiDAR may last <45 min. Mitigation: add low-battery shutdown hook in hw.h (hw_get_battery_voltage); measure full power budget during W5 hardware setup.
RISK-07	LOW	Digital Twin WebSocket latency <i>Responsible: MOD-02</i>	If RPi WiFi is congested, the 10 Hz telemetry feed may lag. Cosmetic issue only — no effect on robot safety. MOD-02 should implement dead-reckoning interpolation to compensate.
RISK-08	MED	obstacle_alert_t bearing_deg accuracy from HC-SR04 <i>Responsible: MOD-06, MOD-05</i>	HC-SR04 sensors have ~30° beam angle; bearing_deg may be inaccurate. MOD-06 and MOD-05 must agree on uncertainty encoding (e.g. NaN = bearing unknown) before W7 integration.